ARKA-Lite

Proyecto final · Upskilling Backend Java

Equipo arka-litos: dos microservicios Spring Boot, sus bases de datos y su plataforma, contados de punta a punta.

RESUMEN · CONTENIDO Y GUION

PARTES

1. **Fundamentos** Primero el qué, después el cómo: el negocio claro y buenas prácticas que sirven en cualquier lenguaje.
2. **Arquitectura** El sistema por fuera: qué piezas hay y cómo trabajan juntas.
3. **Base de datos** Dos bases PostgreSQL, el ORM, la cadena de conexión y las transacciones.
4. **Interconexión de servicios** Cómo se hablan los servicios: síncrono, asíncrono, Outbox e idempotencia.
5. **DevOps: Docker y Kubernetes** Docker empaqueta, Kubernetes administra y el pipeline prueba cada cambio.
6. **Observabilidad** Logs en un solo lugar, trazas entre servicios y salud de cada componente.
7. **Demostración con la UI** La UI ejemplifica en vivo lo que contamos: estados, Outbox, idempotencia y salud.

Generado desde docs/presentation-data/contenido. Cada lámina tiene: lo que se ve, el diagrama detallado cuando lo hay, y el guion: qué decir, qué hacer, si preguntan y para pasar.

Apertura

1. Del endpoint al clúster

Presenta: Desarrollo 1 · orquesta

EN PANTALLA · NÚMEROS GRANDES

- 2 microservicios
- 2 bases PostgreSQL
- 1 comando levanta todo
- 0 avisos perdidos o duplicados

IDEAS CLAVE

- Cómo construimos dos microservicios con sus datos, su plataforma y su operación.

LO QUE SE LLEVA EL STAFF

En 40 minutos, prácticas que pueden aplicar en sus propios sistemas, no solo teoría.

QUÉ DECIR (? = HACER CLIC)

"Buenos días a todos. Somos el equipo arka-litos, del curso Upskilling Backend Java, y hoy les presentamos ARKA-Lite, nuestro proyecto final."

"Somos un equipo de tres áreas, las mismas que tienen ustedes: desarrollo, base de datos e infraestructura, y cada parte la presenta quien la construyó. A mí, que vengo de desarrollo y conozco el negocio de punta a punta, me toca abrir, conectar las partes y cerrar."

"El título lo resume: empezamos escribiendo un endpoint, una sola operación en Java, y terminamos con un sistema completo corriendo en un clúster de Kubernetes, con sus bases de datos, su pipeline y su monitoreo."

"El equipo anterior ya les explicó los fundamentos y la arquitectura. Nosotros les vamos a contar el proyecto desde la práctica, con diagramas y una demostración en vivo, y nos vamos a enfocar en lo que les puede servir en su día a día: cómo se cuidan los datos, cómo se hablan los sistemas y cómo se despliega y se opera todo."

QUÉ HACER (NO SE DICE)

- Antes de empezar, abrir la UI (http://localhost:8088, o el puerto 8088 del Codespace) y comprobar que los dos servicios aparecen en verde. Guía del Codespace: docs/operaciones/codespaces.md.
- Tener abiertas en otras pestañas Grafana (http://localhost:3000) y la carpeta k8s/ del repositorio, por si preguntan.
- Cada diagrama se arma con clics: en el texto, ? CLIC marca el momento de hacer clic.

SI HAY TIEMPO, SE PUEDE AGREGAR

- ARKA-Lite nació como el laboratorio del curso: cada tema que veíamos lo aplicábamos al mismo sistema. Por eso creció por capas: primero un endpoint, después la base de datos, la comunicación entre servicios, los contenedores, el pipeline y la observabilidad.

- Todo lo que van a ver está en un solo repositorio, con su documentación, y se levanta con un solo comando. Si a alguien le interesa revisarlo después, con gusto se lo compartimos.
- Trabajamos como trabajaría un equipo real: cada cambio en una rama, revisado por otra persona y probado por el pipeline antes de entrar. Mezclar áreas nos obligó a ponernos de acuerdo en contratos claros: qué expone cada servicio, qué guarda cada base y cómo se despliega cada pieza.

PARA PASAR

"Este es el recorrido."

---

2. Qué vamos a ver

Presenta: Desarrollo 1 · orquesta

EN PANTALLA · LA RUTA DE LAS PARTES

- 1. Fundamentos (vista general): Primero el qué, después el cómo: el negocio claro y buenas prácticas que sirven en cualquier lenguaje.
- 2. Arquitectura (vista general): El sistema por fuera: qué piezas hay y cómo trabajan juntas.
- 3. Base de datos (a fondo): Dos bases PostgreSQL, el ORM, la cadena de conexión y las transacciones.
- 4. Interconexión de servicios (a fondo): Cómo se hablan los servicios: síncrono, asíncrono, Outbox e idempotencia.
- 5. DevOps: Docker y Kubernetes (a fondo): Docker empaqueta, Kubernetes administra y el pipeline prueba cada cambio.
- 6. Observabilidad (vista general): Logs en un solo lugar, trazas entre servicios y salud de cada componente.
- 7. Demostración con la UI (en vivo): La UI ejemplifica en vivo lo que contamos: estados, Outbox, idempotencia y salud.

IDEAS CLAVE

- Fundamentos y arquitectura: vista general
- Base de datos: a fondo
- Interconexión de servicios: a fondo
- DevOps con Docker y Kubernetes: a fondo
- Observabilidad y demostración en vivo

LO QUE SE LLEVA EL STAFF

Saber desde el inicio dónde vamos a profundizar y qué se van a llevar.

QUÉ DECIR (? = HACER CLIC)

"Todos los equipos presentamos las mismas secciones, pero cada uno profundiza en algunas para no repetirnos. El equipo anterior se enfocó en los fundamentos y en la arquitectura hexagonal, así que esas dos partes las vamos a ver rápido, en vista general."

"? Donde vamos a fondo es en tres temas que tocan directo el trabajo de ustedes: la base de datos, cómo se comunican los servicios sin perder información, y DevOps, con Docker, Kubernetes y el pipeline. ? Después, una mirada general a la observabilidad, ? y cerramos con una demostración en vivo: vamos a tumbar un servicio a propósito para que vean cómo se comporta el sistema."

"Al final de cada parte les dejamos lo que aprendimos y cómo se puede aplicar en cualquier proyecto, también en los que usan procedimientos almacenados."

SI HAY TIEMPO, SE PUEDE AGREGAR

- Las partes generales son cortas a propósito: el equipo anterior ya las explicó. El tiempo lo dedicamos a lo que más valor les deja: los datos, la comunicación entre sistemas y la operación.
- Al final juntamos lo que pueden aplicar desde mañana, aunque sus sistemas usen procedimientos almacenados.

PARA PASAR

“Empecemos por los fundamentos, y por lo primero: el qué.”

1 · Fundamentos

Primero el qué, después el cómo: el negocio claro y buenas prácticas que sirven en cualquier lenguaje.

3. Primero el qué: el negocio en una lámina

Presenta: Desarrollo 1 · orquesta

EN PANTALLA · FLUJO PASO A PASO

- 1. Quien solicita: Registra qué necesita y de qué tipo.
- 2. La envía: Cuando está lista.
- 3. El sistema avisa: A quien debe atenderla.
- 4. Alguien decide: La aprueba o la rechaza.
- Si si se salta un paso en el paso 2: No se permite: Enviar dos veces o decidir sobre un borrador: el sistema explica por qué.

IDEAS CLAVE

- Quien solicita registra la solicitud
- La envía cuando está lista
- El sistema le avisa a quien debe atenderla
- Alguien la aprueba o la rechaza
- Saltarse un paso no se permite

LO QUE SE LLEVA EL STAFF

Pensar primero en el qué y después en el cómo: las reglas claras hacen el código simple.

QUÉ DECIR (? = HACER CLIC)

“Antes de entrar a la tecnología, el aprendizaje más importante de los fundamentos: primero pensamos en el qué y después en el cómo. ¿Qué tiene que pasar, quién lo hace y qué no se puede permitir? Este es el negocio, en una lámina. Alguien registra una solicitud, por ejemplo una incidencia o un pedido de cambio: qué necesita y de qué tipo.”

“? Cuando está lista, la envía. ? En ese momento, el sistema le avisa a quien debe atenderla. ? Y esa persona la aprueba o la rechaza: es la decisión final.”

“? Igual de importante es lo que no se permite: enviar dos veces la misma solicitud, o aprobar una que todavía nadie ha enviado. El sistema no lo hace y explica por qué. Esas son las reglas del negocio, y con ellas claras, el código salió simple: un servicio para las solicitudes y otro para los avisos.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- Antes de escribir una línea de código nos hicimos tres preguntas: qué tiene que pasar, quién hace cada paso y qué no se puede permitir. Las respuestas se volvieron las reglas del sistema.
- Por dentro, esos pasos son cuatro estados: borrador, enviada, aprobada y rechazada. La base de datos también los cuida: la columna estado solo acepta esos cuatro valores.
- Al arrancar, el sistema siempre registra dos solicitudes de ejemplo: INC-001, una incidencia en borrador, y CAM-002, un cambio que se envía. El pipeline usa CAM-002 para comprobar que el servicio arrancó de verdad.

SI PREGUNTAN

¿Por qué los avisos van en otro servicio?

Porque cambian por motivos distintos, y así el trámite sigue funcionando aunque el servicio de avisos esté caído.

PARA PASAR

“Con el qué claro, el cómo: código que se pueda cambiar sin miedo.”

---

4. Código mantenible: buenas prácticas

Presenta: Desarrollo 1 · orquesta

EN PANTALLA · TARJETAS

- Una tarea por pieza: Quien recibe la petición no decide; quien decide no guarda.
- Piezas que se conectan: Cada pieza recibe lo que necesita: se cambia o se prueba sola.
- Reglas en un solo lugar: El negocio vive junto, lejos de la pantalla y de la base.
- Errores con respuesta clara: Siempre el mismo formato: 400, 404 o 409 con su motivo.
- Nombres del negocio: Solicitud, enviar, aprobar: el código se lee como el proceso.
- Cada regla con su prueba: Una prueba automática avisa si un cambio rompió algo.

IDEAS CLAVE

- Una tarea por pieza
- Las piezas se conectan, no se fabrican adentro
- Las reglas viven en un solo lugar
- Errores con respuesta clara
- Nombres del negocio
- Cada regla con su prueba

LO QUE SE LLEVA EL STAFF

Cuando cambia una regla, se toca un solo lugar y una prueba confirma que nada se rompió.

QUÉ DECIR (? = HACER CLIC)

“Con el qué claro, viene el cómo. El equipo anterior ya cubrió los fundamentos, así que nos quedamos con seis prácticas que nos sirvieron y que se pueden usar en cualquier lenguaje, sea Java, C#, Python o PL/SQL. Primera: cada pieza hace una sola tarea; la que recibe la petición no decide, y la que decide no guarda.”

“? Segunda: las piezas se conectan desde afuera, así se pueden cambiar o probar por separado. ? Tercera: las reglas del negocio viven en un solo lugar, no repetidas en la pantalla y en la base. ? Cuarta: los errores responden siempre igual: un 400 si faltan datos, un 404 si algo no existe, un 409 si la regla no lo permite.”

“? Quinta: los nombres son los del negocio, así el código se lee como el proceso. ? Y sexta: cada regla tiene una prueba automática que avisa si un cambio rompió algo. El resultado: cuando cambia una regla, se toca un solo lugar.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- Un ejemplo concreto: el controlador de solicitudes no decide nada; solo convierte el JSON y llama al servicio. Si mañana cambia la regla de envío, se toca la clase Solicitud y su prueba, nada más.
- Lo de las piezas que se conectan desde afuera tiene un nombre: inyección de dependencias. En ARKA-Lite todo llega por el constructor, y por eso en las pruebas cambiamos la base real por una en memoria sin tocar el negocio.

SI PREGUNTAN

¿Esto no hace el código más largo?

Un poco más de piezas, pero cada una es corta y se entiende sola. El costo se recupera en el primer cambio.

PARA PASAR

“Y como somos varios trabajando sobre el mismo código, también necesitamos orden.”

2 · Arquitectura

El sistema por fuera: qué piezas hay y cómo trabajan juntas.

5. El sistema de un vistazo

Presenta: Desarrollo 2

EN PANTALLA · CAPAS DEL SISTEMA

- Experiencia: Persona: Usa el navegador; arka-lite-ui: La pantalla · reenvía /api
- Servicios: servicio-solicitudes: El trámite y sus estados; servicio-notificaciones: Los avisos, sin duplicados
- Datos: db-solicitudes: PostgreSQL 17 · solicitud y evento; db-notificaciones: PostgreSQL 17 · evento_procesado
- Plataforma: Compose o Kubernetes: Dónde corre; GitHub Actions: Prueba y publica; Loki y Grafana: Logs en un solo lugar

IDEAS CLAVE

- La UI es la puerta de entrada

- Dos servicios, cada uno dueño de su base
- Entre servicios solo viajan datos
- La plataforma corre, entrega y observa

LO QUE SE LLEVA EL STAFF

Cada capa se puede cambiar sin romper las otras.

QUÉ DECIR (? = HACER CLIC)

"Gracias. ARKA-Lite no es una aplicación grande: son piezas pequeñas, cada una con una responsabilidad. Arriba está la experiencia: la persona usa la interfaz, y la interfaz no guarda nada, solo les pasa las peticiones a los servicios."

"? En el medio, los dos microservicios: solicitudes lleva el trámite y sus estados; notificaciones crea los avisos. ? Debajo, cada uno con su propia base PostgreSQL; ninguno lee la base del otro. ? Y abajo, la plataforma: dónde corre, cómo se prueba cada cambio y dónde se ven los logs."

SI HAY TIEMPO, SE PUEDE AGREGAR

- La interfaz es delgada a propósito: no guarda nada ni toma decisiones. Si mañana se cambia por otra, los servicios no se enteran.
- La plataforma es la misma para los dos servicios: el mismo Compose, el mismo pipeline y el mismo lugar para los logs. Eso baja mucho el costo de tener dos servicios en lugar de uno.

SI PREGUNTAN

¿Por qué no comparten una sola base de datos?

Para que cada servicio sea dueño de sus datos y pueda cambiar sin romper al otro.

PARA PASAR

"Ahora veamos cómo trabajan juntas esas piezas en la práctica."

---

6. Cómo funciona en la práctica

Presenta: Desarrollo 2

EN PANTALLA · EL SISTEMA EN LA PRÁCTICA

- 1. Persona -> La UI: usa
- 2. La UI -> Solicitudes: crea, envía y decide
- 3. Solicitudes -> db-solicitudes: guarda
- 4. Solicitudes ? Notificaciones: avisa después (SolicitudEnviada: eventoId, id y tipo)
- 5. Notificaciones -> db-notificaciones: anota lo recibido
- 6. La UI -> Notificaciones: consulta los avisos
- Ninguna flecha une las dos bases: cada servicio es dueño de sus datos.

IDEAS CLAVE

- La persona usa la UI
- La UI le pide el trabajo a cada servicio
- Cada servicio guarda en su propia base
- Solicitudes avisa con un evento: un dato, no una pieza

LO QUE SE LLEVA EL STAFF

Los sistemas se comunican con mensajes claros, no compartiendo tablas.

QUÉ DECIR (? = HACER CLIC)

“Ahora, cómo trabajan juntas en la práctica. Sigamos los números. ? Uno: la persona usa la interfaz. ? Dos: cuando crea, envía o decide una solicitud, la interfaz se lo pide al servicio de solicitudes. ? Tres: solicitudes lo guarda en su propia base.”

“? Cuatro, el paso más importante: cuando una solicitud se envía, solicitudes le avisa a notificaciones. Fíjense en que esa flecha no es una pieza del sistema: es un dato que viaja, un mensaje pequeño con el número del evento, la solicitud y su tipo.”

“? Cinco: notificaciones crea el aviso y anota que ya recibió ese evento. ? Seis: la interfaz le consulta a notificaciones para mostrar los avisos.”

“? Y lo que no hay también dice mucho: ninguna flecha une las dos bases. Si un servicio necesita algo del otro, se lo pide; nunca entra a su base.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- El mensaje que viaja entre servicios lleva solo tres datos: el número del evento, el identificador de la solicitud y su tipo. Mientras más pequeño el mensaje, más fácil es que los dos lados se pongan de acuerdo.
- Ese contrato está copiado en los dos servicios. Si un día cambia, se cambia en los dos al mismo tiempo: es lo único que comparten.

SI PREGUNTAN

¿Qué pasa si notificaciones está caído en el paso cuatro?
Solicitudes sigue funcionando y el aviso espera guardado. Lo vemos en la parte de interconexión y en la demostración.

PARA PASAR

“¿Y cómo está ordenado cada servicio por dentro?”

3 · Base de datos

Dos bases PostgreSQL, el ORM, la cadena de conexión y las transacciones.

7. Dos bases, una por servicio

Presenta: Base de datos

EN PANTALLA · NÚMEROS GRANDES

- 2 bases de datos: db-solicitudes y db-notificaciones: una por servicio.
- 17 PostgreSQL: El mismo motor y la misma versión en las dos.
- 0 JOIN entre bases: Ni consultas ni llaves foráneas que crucen de una a otra.
- 1 disco por base: En Kubernetes, los datos sobreviven aunque el contenedor se reinicie.

IDEAS CLAVE

- Dos bases de datos, una por servicio
- El mismo motor: PostgreSQL 17
- Ninguna consulta cruza de una base a otra
- En Kubernetes, cada base tiene su disco

LO QUE SE LLEVA EL STAFF

Cada base cambia a su ritmo, sin coordinar despliegues con otros equipos.

QUÉ DECIR (? = HACER CLIC)

"Gracias. Como les contó mi compañero de arquitectura, el sistema tiene dos bases de datos, una para cada microservicio: db-solicitudes y db-notificaciones. ? Las dos están montadas en PostgreSQL, versión diecisiete."

"¿Por qué dos y no una? Porque cada servicio es dueño de sus datos. Solicitudes guarda las solicitudes y los avisos que tiene pendientes; notificaciones guarda qué avisos ya recibió. Si mañana una cambia su estructura, la otra ni se entera."

"? Eso tiene una consecuencia que para quienes trabajamos con bases de datos pesa mucho: no hay ningún JOIN entre las dos bases, ni llaves foráneas que crucen. Si un servicio necesita algo del otro, se lo pregunta por HTTP."

"? Y cada base corre en su propio contenedor. En Kubernetes, además, tiene su disco persistente: los datos sobreviven aunque el contenedor se reinicie."

SI HAY TIEMPO, SE PUEDE AGREGAR

- En nuestra computadora, la base de notificaciones se publica en el puerto 5433 para consultarla con cualquier herramienta; la de solicitudes solo se ve dentro de la red de Docker. Lo que no se publica, no se alcanza desde afuera.
- En Kubernetes, cada base tiene su disco persistente: si el contenedor se reinicia, los datos siguen ahí. Solo se perderían si se borra el disco.

SI PREGUNTAN

¿Y si necesito un reporte que junte datos de las dos?

Se arma fuera de los servicios, por ejemplo con una réplica o un proceso de reportes que lea de ambas. Lo que no se hace es que un servicio lea la base del otro.

SI PREGUNTAN

¿No es más difícil administrar dos bases?

Son dos bases pequeñas con una sola responsabilidad cada una. Se respaldan y se monitorean como cualquier PostgreSQL, y la caída de una no tumba a la otra.

PARA PASAR

"Veamos qué hay dentro de cada una."

---

8. Del procedimiento almacenado al ORM

Presenta: Base de datos

EN PANTALLA · DOS CAMINOS, LADO A LADO

- Hoy: procedimientos almacenados: Pantalla: La persona pulsa Enviar. -> [clic] Aplicación: Solo pasa los parámetros. -> [EXEC sp_enviar @id] Base de datos: El procedimiento decide y guarda. (aquí vive la regla)
- ARKA-Lite: un ORM: Pantalla: La persona pulsa Enviar. -> [POST /enviar] Servicio Java: Decide, con su prueba. (aquí vive la regla) -> [save(…)] Hibernate: Escribe el SQL. -> [UPDATE + INSERT] PostgreSQL: Guarda y cuida llaves e índices.

IDEAS CLAVE

- Con SP, la regla vive en la base
- Con ORM, la regla vive en el código
- Hibernate escribe el SQL de todos los días
- La base sigue cuidando datos, llaves e índices

LO QUE SE LLEVA EL STAFF

Reglas que se prueban en segundos y una base que sigue cuidando los datos.

QUÉ DECIR (? = HACER CLIC)

"Esta es la gran diferencia con muchas de las aplicaciones que gestionamos hoy, y no es la base de datos: es dónde vive la regla del negocio. Arriba, el enfoque que conocemos: la persona pulsa enviar, la aplicación solo pasa parámetros con un EXEC, y el procedimiento almacenado decide y guarda. La regla vive en la base."

"? Abajo, ARKA-Lite: la regla vive en el servicio Java, junto con su prueba. Cuando decide que sí, Hibernate, que es el ORM, escribe el UPDATE y el INSERT. La base sigue guardando y cuidando llaves, restricciones e índices. Lo único que se movió es la decisión."

SI HAY TIEMPO, SE PUEDE AGREGAR

- No estamos diciendo que los procedimientos estén mal: son la herramienta correcta para procesos masivos y reportes. Lo que cambia es dónde ponemos las reglas que más cambian y que más necesitamos probar.
- Una ventaja que no se ve a primera vista: con la regla en el código, cada cambio pasa por revisión, pruebas automáticas y pipeline, igual que cualquier otra línea del sistema.

SI PREGUNTAN

¿Y el rendimiento? ¿El SQL que genera Hibernate es bueno?

Para guardar y buscar por llave, sí. Para consultas complejas se puede escribir SQL propio; además, los índices y las restricciones siguen siendo trabajo de la base.

SI PREGUNTAN

¿Hay que migrar todos los procedimientos?

No. Se puede empezar por las reglas que más cambian; los reportes y los procesos masivos pueden seguir en la base.

PARA PASAR

"Mi compañero de desarrollo se los muestra con una regla real, escrita de las dos formas."

---

9. ¿Dónde está la cadena de conexión?

Presenta: Desarrollo 3

EN PANTALLA · DE DÓNDE SALE CADA VALOR

- Centro: El servicio al arrancar: application.properties solo tiene marcadores. (url=${DB_URL}; username=${DB_USER}; password=${DB_PASSWORD})
- Viene de Docker Compose: compose.yaml, en cada servicio: DB_URL -> db-solicitudes:5432
- Viene de Kubernetes · Deployment: La dirección y el usuario: DB_URL · DB_USER
- Viene de Kubernetes · Secret: La clave, guardada aparte: DB_PASSWORD <- Secret arka-postgres
- Llega a db-solicitudes: PostgreSQL 17 por pool de conexiones

- Nunca: La imagen Docker: no lleva ni dirección ni clave. La misma imagen sirve en cualquier ambiente.

IDEAS CLAVE

- El código solo tiene marcadores: ${DB_URL}
- Compose los llena con variables de entorno
- Kubernetes: la dirección en el Deployment y la clave en un Secret
- La imagen no lleva ni dirección ni clave

LO QUE SE LLEVA EL STAFF

Las claves fuera del código: rotarlas o cambiar de servidor no requiere recompilar.

QUÉ DECIR (? = HACER CLIC)

"Esta es la pregunta que siempre nos hacen: ¿dónde está la cadena de conexión? Empecemos por el centro: el servicio, al arrancar, lee su archivo de configuración, y ese archivo no tiene la dirección ni la clave. Solo tiene marcadores: DB_URL, DB_USER y DB_PASSWORD."

"¿Quién llena esos marcadores? El ambiente donde corre. ? En Docker Compose, las variables se definen para cada servicio en el archivo compose.yaml. ? En Kubernetes, la dirección y el usuario van en el Deployment, ? y la clave se guarda aparte, en un Secret: un objeto del clúster hecho para datos sensibles, que el contenedor recibe como variable al arrancar."

"? Con esos valores, el servicio abre su pool de conexiones contra la base."

"? ¿Y la imagen Docker? No lleva ni dirección ni clave. Es la misma imagen en desarrollo, en pruebas y en producción. Cambiar de servidor de base de datos o rotar una clave no obliga a recompilar ni a tocar el código."

SI HAY TIEMPO, SE PUEDE AGREGAR

- Un detalle honesto: el archivo de configuración trae un valor por defecto para trabajar en la computadora de cada uno. En un ambiente real nunca se usa, porque ahí siempre llegan las variables.
- En el pipeline, la clave de la base de pruebas cambia en cada ejecución: se arma con el número de la corrida. Ni siquiera la base temporal tiene una clave fija.

SI PREGUNTAN

¿El Secret no está en el repositorio?

En este laboratorio sí, con una clave de ejemplo, para que cualquiera lo levante. En un ambiente real el Secret se crea aparte, o lo alimenta un gestor de secretos, y nunca se versiona.

SI PREGUNTAN

¿Y si no usamos Kubernetes?

La misma idea funciona con variables de entorno del servidor o con un gestor de secretos. Lo importante es que la clave no viva en el código ni en el paquete que se despliega.

SI PREGUNTAN

¿Quién puede ver la clave en Kubernetes?

Quien tenga permiso de leer Secrets en ese espacio del clúster; ese acceso se controla con roles.

PARA PASAR

"Con la conexión resuelta, mi compañero de base de datos les cuenta qué pasa cuando se guarda: la transacción."

4 · Interconexión de servicios

Cómo se hablan los servicios: síncrono, asíncrono, Outbox e idempotencia.

10. Síncrono o asíncrono

Presenta: Desarrollo 4

EN PANTALLA · DOS MANERAS, LADO A LADO

- Síncrono: La UI llama y espera que se complete el proceso.; Respuesta inmediata; Si el servicio se cae, la persona ve error
- Asíncrono: Servicio solicitudes envía evento a servicio notificaciones y continúa con su proceso; Respuesta puede demorar; Si notificaciones se cae, el evento espera

IDEAS CLAVE

- Síncrono: la UI llama y espera
- Asíncrono: solicitudes envía el evento y sigue
- Síncrono responde ya; asíncrono puede demorar
- Si notificaciones se cae, el evento espera

Comparación: Síncrono es una llamada: me quedo en la línea hasta que me responden. Asíncrono es un correo: lo envío y sigo con lo mío.

LO QUE SE LLEVA EL STAFF

La caída de un sistema secundario no detiene a quien está atendiendo al usuario.

QUÉ DECIR (? = HACER CLIC)

"Hay dos formas de comunicarse. La síncrona es como una llamada telefónica: la interfaz llama y espera a que el proceso se complete. La respuesta es inmediata, y por eso la usamos para crear, aprobar o rechazar: la persona necesita saber el resultado en ese momento. El costo es que, si el servicio se cae, la persona ve el error."

"? La asíncrona es como un correo: el servicio de solicitudes envía el evento a notificaciones y continúa con su proceso, sin esperar. La respuesta puede demorar unos segundos, pero si notificaciones se cae, el evento no se pierde: espera hasta que el servicio vuelva."

"La regla que aprendimos: lo que la persona necesita ver, síncrono; lo que puede esperar, asíncrono. Y esto pasa también en sus sistemas de hoy: un procedimiento que llama a otro sistema y espera su respuesta falla si el otro está caído."

SI HAY TIEMPO, SE PUEDE AGREGAR

- Un ejemplo de la vida diaria: cuando pagamos con tarjeta, la aprobación es síncrona, porque necesitamos saber si pasó; el correo con el comprobante es asíncrono, porque puede llegar un minuto después.
- La pregunta que nos hacemos en cada caso es: si el otro sistema está caído, ¿la persona tiene que enterarse? Si la respuesta es no, va asíncrono.

SI PREGUNTAN

¿Por qué no hacerlo todo asíncrono?

Porque quien crea o aprueba una solicitud necesita saber en ese momento si se pudo. Lo asíncrono es

para lo que puede esperar.

PARA PASAR

“¿Y cómo logramos que el evento espere sin perderse? Con el patrón Outbox.”

---

11. Patrón Outbox: guardar primero, avisar después

Presenta: Desarrollo 4

EN PANTALLA · FLUJO PASO A PASO

- 1. Enviar: Solicitud ENVIADA + evento pendiente, en una transacción.
- 2. Bandeja de salida: El evento espera en la tabla evento.
- 3. Relevador: Cada 5 s entrega los pendientes, hasta 3 intentos.
- 4. Notificaciones: Lo recibe y el evento queda procesado.
- Si si falla en el paso 3: Sigue pendiente: Mismo eventoId; se reintenta en el siguiente ciclo.

IDEAS CLAVE

- Solicitud ENVIADA y evento pendiente, juntos
- Si una falla, no se guarda ninguna
- El relevador entrega cada 5 s, con 3 intentos
- Si falla, sigue pendiente con el mismo eventoId

Comparación: Como dejar una carta en la bandeja de salida: aunque el mensajero no pase hoy, la carta no se pierde.

LO QUE SE LLEVA EL STAFF

Ningún aviso se pierde, aunque el otro sistema esté caído. Y se puede aplicar también con procedimientos y un job.

QUÉ DECIR (? = HACER CLIC)

“Pensemos en el problema. Si al enviar una solicitud llamáramos directamente a notificaciones y justo estuviera caído, la solicitud quedaría enviada y nadie se enteraría. Por eso usamos el patrón Outbox, o bandeja de salida, que se apoya en la transacción que vimos en la parte de datos.”

“Al enviar, en una sola transacción se guardan la solicitud como enviada y su aviso pendiente. O se guardan las dos cosas, o ninguna, y la persona recibe su respuesta enseguida. ? El aviso queda esperando en la tabla evento, como una carta en la bandeja de salida.”

“? Después, un proceso que llamamos el relevador revisa la bandeja cada cinco segundos y entrega los pendientes, con hasta tres intentos. ? Si notificaciones lo recibe, la fila se marca como procesada. ? Si falla, la fila sigue pendiente, con el mismo número de evento, y se reintenta en el siguiente ciclo.”

“Esto se puede aplicar hoy mismo en un sistema con procedimientos almacenados: en lugar de llamar a otro sistema dentro del procedimiento, se anota el aviso en una tabla de pendientes y un job lo entrega.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- El relevador hace hasta tres intentos por ciclo, esperando un poco más entre cada uno. Si los tres fallan, no pasa nada grave: la fila sigue pendiente y el siguiente ciclo, cinco segundos después, vuelve a intentar.
- El número del evento se genera una sola vez, al guardar. Por eso cada reintento manda exactamente el

mismo número, y del otro lado se puede reconocer como repetido.

- En un sistema con procedimientos se traduce así: una tabla de pendientes con una columna procesado, un INSERT dentro de la misma transacción del procedimiento y un job que recorre los pendientes.

SI PREGUNTAN

¿Que el envío responda bien significa que el aviso ya existe?
No. Significa que la solicitud y el evento quedaron guardados; el aviso llega después.

SI PREGUNTAN

¿Qué pasa si se cae solicitudes con avisos pendientes?
Nada se pierde: están en la base. Cuando el servicio vuelve, el relevador los encuentra y los entrega.

PARA PASAR

“Pero si se reintenta, ¿no puede llegar el aviso dos veces?”

5 · DevOps: Docker y Kubernetes

Docker empaqueta, Kubernetes administra y el pipeline prueba cada cambio.

12. Docker y Kubernetes: no compiten, se complementan

Presenta: Infraestructura 1

EN PANTALLA · DOS MANERAS, LADO A LADO

- Docker: empaqueta: Mete la app con Java y todo lo que usa; Crea una imagen que no cambia; Corre igual en una laptop o en un servidor; Compose levanta varias cajas juntas
- Kubernetes: administra: Decide dónde corre cada caja; Si una se cae, levanta otra; Le da a cada servicio un nombre fijo; Puede correr más copias si hay carga

IDEAS CLAVE

- Docker empaqueta la app con todo lo que necesita
- Kubernetes decide dónde corre y la vigila
- La misma imagen sirve en los dos
- Uno sin el otro queda a medias

Comparación: Como el transporte de carga: Docker es el contenedor estándar y Kubernetes, el puerto que los acomoda, los vigila y reemplaza el que se daña.

LO QUE SE LLEVA EL STAFF

El mismo paquete del laptop al servidor, y un sistema que se cuida solo.

QUÉ DECIR (? = HACER CLIC)

“Docker y Kubernetes muchas veces se presentan como si uno reemplazara al otro, y no es así: no compiten, se complementan. Docker resuelve el clásico “en mi máquina funciona”: empaqueta la aplicación con Java y todo lo que necesita en una imagen, un paquete que no cambia y que corre igual en una laptop o en un servidor.”

“? Kubernetes empieza donde termina Docker. No empaqueta nada: toma esas imágenes y las administra. Decide dónde corre cada contenedor, lo vigila y, si uno se cae, levanta otro.”

“Piénsenlo como el transporte de carga: Docker es el contenedor estándar y Kubernetes es el puerto que los acomoda y reemplaza el que se daña. En nuestra computadora, Docker Compose levanta los ocho contenedores del sistema con un solo comando; en el clúster, esas mismas imágenes las administra Kubernetes, sin recompilar nada.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- La imagen de cada servicio se construye en dos etapas: una con Maven para compilar y otra solo con Java para correr. Así la imagen final es más liviana y no lleva herramientas de compilación.
- Además, el contenedor corre con un usuario sin privilegios: si alguien comprometiera la aplicación, no sería administrador del contenedor.

SI PREGUNTAN

¿Docker es una máquina virtual?

No. Es más liviano: comparte el sistema operativo y solo aísla la aplicación y lo que necesita.

SI PREGUNTAN

¿Necesitamos Kubernetes para todo?

No. Para un sistema pequeño basta Docker Compose en un servidor. Kubernetes vale la pena cuando hay varios servicios, varias copias o hay que recuperarse solo.

PARA PASAR

“Veamos cómo nace un contenedor.”

---

13. Cuando un Pod se cae

Presenta: Infraestructura 2

EN PANTALLA · QUÉ PASA CUANDO FALLA

- Falla: Se cae el Pod de solicitudes: Un error, un nodo reiniciado o un kubectl delete.
- 1. [0 s] Detecta: Falta 1 de 1 copias.
- 2. [+1 s] Recrea: Un Pod nuevo con la misma imagen.
- 3. [+segundos] Espera: initContainers: base y notificaciones.
- 4. [Ready] Listo: La readinessProbe responde.
- Resultado: Sin intervención manual: Los eventos pendientes siguen en la base y se entregan.

IDEAS CLAVE

- El Deployment detecta que falta una copia
- Crea un Pod nuevo
- Espera a la base y a su salud
- Vuelve a recibir tráfico, sin intervención

LO QUE SE LLEVA EL STAFF

Un sistema que se recupera solo, sin que nadie se despierte a las tres de la mañana.

QUÉ DECIR (? = HACER CLIC)

"Ya vimos que Kubernetes compara todo el tiempo lo que hay con lo que pedimos. Veamos qué pasa cuando se cae el Pod de solicitudes, por un error, porque se reinició el servidor o porque alguien lo borró."

"? En el mismo instante, el Deployment detecta que falta una copia. ? En un segundo, crea un Pod nuevo con la misma imagen. ? El Pod nuevo espera a que su base y notificaciones respondan, ? y solo cuando su prueba de salud contesta, vuelve a recibir tráfico."

"? Nadie intervino. Y como los avisos pendientes están en la base, no en memoria, nada se pierde mientras tanto. Aquí se juntan las tres partes que vimos a fondo: la base guarda, el Outbox espera y Kubernetes repara."

QUÉ HACER (NO SE DICE)

- Mostrar: kubectl delete pod -l app=servicio-solicitudes y luego kubectl get pods -w. Debe verse: Aparece un Pod nuevo que vuelve a estar Ready.

SI HAY TIEMPO, SE PUEDE AGREGAR

- Se demuestra en un minuto: se borra el Pod con kubectl delete y, mirando la lista de Pods, aparece uno nuevo que en unos segundos vuelve a estar listo.
- Si lo que se cae es la base, pasa lo mismo: su Pod se recrea y los datos siguen en el disco persistente.

SI PREGUNTAN

¿Y si se cae la base?

Su Pod también se recrea y los datos siguen en el disco persistente; los servicios esperan a que vuelva.

PARA PASAR

"Esto es lo que aprendimos de DevOps."

6 · Observabilidad

Logs en un solo lugar, trazas entre servicios y salud de cada componente.

14. Todos los logs, en un solo lugar

Presenta: Infraestructura 1

EN PANTALLA · FLUJO PASO A PASO

- 1. Servicios: Logs JSON con traceId y eventoId.
- 2. Promtail: Los recoge de cada contenedor.
- 3. Loki: Los guarda, etiquetados por servicio.
- 4. Grafana: Una búsqueda para todo el sistema.

IDEAS CLAVE

- Cada servicio escribe sus logs en JSON
- Promtail los recoge de todos los contenedores
- Loki los guarda por servicio

- Grafana los busca por traceId o eventoId

Comparación: Como la caja negra de un avión: registra lo que pasó, para revisarlo después.

LO QUE SE LLEVA EL STAFF

Diagnosticar un error entre sistemas en minutos, con una sola búsqueda.

QUÉ DECIR (? = HACER CLIC)

"Gracias. La observabilidad la vemos de forma general. Con varios servicios, cada uno tiene sus logs, y si algo falla no queremos entrar contenedor por contenedor. Cada servicio escribe sus logs en JSON, con un formato estándar. ? Promtail los recoge de todos los contenedores, ? Loki los guarda etiquetados por servicio ? y Grafana nos deja buscarlos en un solo lugar."

"Como cada llamada lleva su traceId, con una sola búsqueda vemos el viaje completo de un evento: cuándo lo publicó el relevador y cuándo lo consumió notificaciones. Y cada servicio expone su salud, que la interfaz muestra en verde o en rojo; lo van a ver en la demostración."

QUÉ HACER (NO SE DICE)

- Mostrar: Grafana ? Explore ? Loki: {service="servicio-notificaciones"} |= "evento". Debe verse: Las líneas "Consume evento" y "aviso creado" con el número del evento.

SI HAY TIEMPO, SE PUEDE AGREGAR

- Para buscar en Grafana basta una consulta como: los logs del servicio de notificaciones que contengan la palabra evento. En segundos aparecen las líneas con el número del evento.
- Los logs salen en un formato estándar, ECS, sin tocar el código del negocio: es una línea de configuración de Spring Boot.

PARA PASAR

"Esto es lo que aprendimos de observar el sistema."

7 · Demostración con la UI

La UI ejemplifica en vivo lo que contamos: estados, Outbox, idempotencia y salud.

15. En vivo: si notificaciones se cae

Presenta: Desarrollo 2

EN PANTALLA · QUÉ PASA CUANDO FALLA

- Falla: notificaciones está caído: docker compose stop servicio-notificaciones
- 1. [0 s] Enviar: Responde bien: solicitud ENVIADA y evento guardado.
- 2. [cada 5 s] Reintenta: 3 intentos por ciclo; el evento sigue pendiente.
- 3. [en la UI] Se ve: "Notificaciones no responde" y 1 en camino.
- 4. [al volver] Entrega: Mismo eventoId: se crea el aviso.
- Resultado: 1 aviso, 0 perdidos: Sin intervención y, al reentregar, sin duplicados.

IDEAS CLAVE

- Enviar responde bien aunque notificaciones esté caído
- El evento queda pendiente en la bandeja
- El relevador reintenta cada 5 segundos
- Al volver, llega un solo aviso

LO QUE SE LLEVA EL STAFF

Ver con sus propios ojos que una caída no hace perder ni duplicar información.

QUÉ DECIR (? = HACER CLIC)

“Detenemos notificaciones a propósito; en Salud aparece en rojo. ? Ahora enviamos una solicitud. La respuesta es buena: la solicitud queda enviada y su evento queda guardado. La persona no ve ningún error, porque enviar no depende del otro servicio. Eso es lo asíncrono.”

“? Mientras tanto, el relevador intenta entregar el evento cada cinco segundos y falla, pero no lo pierde. ? La pantalla de Enviadas lo muestra: notificaciones no responde, y hay un evento en camino.”

“? Levantamos notificaciones, y en el siguiente ciclo el evento se entrega con el mismo número: aparece exactamente un aviso. Eso es el Outbox. ? Y para cerrar, reentregamos ese mismo evento a propósito: notificaciones lo reconoce y no crea otro aviso. Cero perdidos y cero duplicados, en vivo.”

QUÉ HACER (NO SE DICE)

- Antes: docker compose stop servicio-notificaciones. Mostrar Salud en rojo (DOWN).
- Crear y enviar DEMO-01; mostrar en Enviadas el aviso amarillo “Notificaciones no responde” y “1 en camino”. No hace falta recargar: la pantalla se actualiza sola.
- docker compose start servicio-notificaciones y esperar unos segundos: el evento pasa a entregado y aparece 1 aviso.
- En Enviadas, pulsar Reentregar sobre ese evento: aparece “El aviso no se duplicó” y Avisos duplicados sigue en 0.
- Los clics de la lámina acompañan lo que pasa en la UI; si la demostración en vivo falla, contarla con la lámina.
- Mostrar: Detener notificaciones, enviar DEMO-01, mirar Enviadas, volver a iniciar notificaciones y reentregar el evento. Debe verse: Evento en camino mientras está caído; al volver, entregado y 1 aviso; al reentregar, 0 duplicados.

SI HAY TIEMPO, SE PUEDE AGREGAR

- Si la red de la sala falla, la demostración se puede contar con esta misma lámina: cada clic muestra lo que se vería en la pantalla.
- Si una operación que cambia datos tarda demasiado, la interfaz no la repite sola, porque el cambio pudo haberse guardado aunque la respuesta no llegara.

SI PREGUNTAN

¿Cuánto tiempo puede esperar un evento pendiente?

Indefinidamente: está en la base. Se entrega en el primer ciclo después de que notificaciones vuelve.

PARA PASAR

“Esto es lo que aprendimos de la demostración.”

Valor y cierre

16. El valor agregado

Presenta: Desarrollo 1 · orquesta

EN PANTALLA · TABLA DE TRES COLUMNAS

- Pilar: Datos consistentes · Qué construimos: Transacción local, Outbox e idempotencia · Valor para el equipo: 0 avisos perdidos o duplicados ante caídas
- Pilar: Conexión segura · Qué construimos: Cadena de conexión y claves fuera del código · Valor para el equipo: Cambiar de servidor o de clave sin recompilar
- Pilar: Reproducibilidad · Qué construimos: Docker, Compose y Kubernetes · Valor para el equipo: 1 comando y la misma imagen en todo ambiente
- Pilar: Calidad · Qué construimos: Pruebas automáticas y pipeline en cada cambio · Valor para el equipo: Errores detectados en minutos, antes de integrar
- Pilar: Trazabilidad · Qué construimos: Logs centralizados, traceId e imagen por commit · Valor para el equipo: Diagnóstico en minutos y versiones rastreables

IDEAS CLAVE

- Datos consistentes ante fallas
- Conexiones fuera del código
- Entornos reproducibles
- Calidad en cada cambio
- Trazabilidad de punta a punta

LO QUE SE LLEVA EL STAFF

Cinco pilares que cualquier equipo puede adoptar.

QUÉ DECIR (? = HACER CLIC)

"Gracias. Si tuviéramos que resumir el valor en una lámina, sería esta. Datos consistentes: ante una caída, no se pierde ni se duplica un aviso. ? Conexión segura: la cadena de conexión y las claves viven fuera del código."

"? Reproducibilidad: un comando y la misma imagen en cualquier ambiente. ? Calidad: pruebas automáticas en cada cambio, antes de integrarlo. ? Y trazabilidad: logs en un solo lugar e imágenes rastreables hasta su commit."

SI HAY TIEMPO, SE PUEDE AGREGAR

- Si alguien pregunta por números: cero avisos perdidos o duplicados en las pruebas de caída, un comando para levantar los ocho contenedores y una imagen rastreable por cada commit.

PARA PASAR

"Y lo más importante: qué pueden aplicar ustedes desde mañana."

17. Lo que pueden aplicar desde mañana

Presenta: Desarrollo 1 · orquesta

EN PANTALLA · TABLA DE TRES COLUMNAS

- Práctica: Cadena de conexión fuera del código · Cómo se ve en sus sistemas de hoy: Variables de entorno o un gestor de secretos en el servidor · Qué evita: Claves en el repositorio y recompilar por cambiar de servidor
- Práctica: Guardar y avisar en la misma transacción · Cómo se ve en sus sistemas de hoy: Una tabla de pendientes que llena el procedimiento y un job que la entrega · Qué evita: Avisos perdidos cuando el otro sistema está caído
- Práctica: Idempotencia en lo que se reintenta · Cómo se ve en sus sistemas de hoy: Una tabla de procesados con llave única por número de operación · Qué evita: Pagos, correos o registros duplicados
- Práctica: Un identificador que cruce sistemas · Cómo se ve en sus sistemas de hoy: El mismo número en el log de la aplicación, del procedimiento y del otro sistema · Qué evita: Horas buscando un error entre sistemas
- Práctica: El mismo paquete en todos los ambientes · Cómo se ve en sus sistemas de hoy: Contenedores para aplicaciones y bases de prueba · Qué evita: El clásico “en mi máquina funciona”

IDEAS CLAVE

- Sacar la cadena de conexión del código
- Guardar y avisar en la misma transacción
- Hacer idempotente lo que se reintenta
- Un identificador que cruce sistemas
- El mismo paquete en todos los ambientes

LO QUE SE LLEVA EL STAFF

Cinco prácticas concretas que se pueden empezar a aplicar sin cambiar de tecnología.

QUÉ DECIR (? = HACER CLIC)

“Esta es la lámina que más nos importa que se lleven. Nada de esto exige reescribir sus sistemas; todo funciona también con procedimientos almacenados. Primero: saquen la cadena de conexión del código; usen variables de entorno o un gestor de secretos.”

“? Segundo: si un procedimiento tiene que avisarle a otro sistema, no lo llamen ahí mismo. Anoten el aviso en una tabla de pendientes, en la misma transacción, y que un job lo entregue. Ese es el Outbox. ? Tercero: todo lo que se reintenta, un pago, un correo, un registro, debe ser idempotente: una tabla de procesados con llave única.”

“? Cuarto: un mismo identificador en todos los logs, para seguir una operación de un sistema a otro. ? Y quinto: el mismo paquete en todos los ambientes, para que lo que se prueba sea lo que se despliega.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- El orden que recomendamos: primero la cadena de conexión, porque es un cambio pequeño que baja un riesgo real; después la tabla de pendientes en el proceso que más falla hoy; y luego la idempotencia en lo que se reintenta.
- Ninguna de estas prácticas exige Java, microservicios ni Kubernetes. Son decisiones de diseño que se

pueden tomar en el sistema que ya tienen.

SI PREGUNTAN

¿Por dónde empezamos?

Por la cadena de conexión: es el cambio más pequeño y baja un riesgo real de seguridad. Después, la tabla de pendientes en el proceso que más falle hoy.

PARA PASAR

"Si nos llevamos solo tres ideas, que sean estas."

---

18. Lo que nos llevamos

Presenta: Desarrollo 1 · orquesta

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 0 avisos perdidos o duplicados
- Métrica: 1 comando para levantar todo
- Diseñar para la falla: Guardar primero, reintentar e ignorar duplicados. Valor: Sistemas que se recuperan solos.
- Automatizar la confianza: Pruebas, pipeline, imágenes y estado declarado. Valor: Cambiar rápido sin miedo a romper.
- Hacer visible el sistema: Logs, trazas, salud, Swagger y la UI. Valor: Diagnosticar y explicar en minutos.

IDEAS CLAVE

- Diseñar para la falla
- Automatizar la confianza
- Hacer visible el sistema

LO QUE SE LLEVA EL STAFF

Tres ideas para recordar al salir de la sala.

QUÉ DECIR (? = HACER CLIC)

"Tres ideas para cerrar. ? Diseñar para la falla: en un sistema distribuido algo siempre va a fallar, y la diferencia está en cómo reacciona. ? Automatizar la confianza: pruebas, pipeline e infraestructura declarada para cambiar rápido sin miedo. ? Y hacer visible el sistema: lo que no se ve no se puede diagnosticar ni explicar."

SI HAY TIEMPO, SE PUEDE AGREGAR

- Si se llevan una sola frase: diseñen pensando en qué pasa cuando algo falla, no solo cuando todo sale bien. Y antes de pensar en el cómo, tengan claro el qué.

PARA PASAR

"Gracias por su atención."

---

19. Gracias

Presenta: Desarrollo 1 · orquesta

EN PANTALLA

- Equipo arka-litos
- Proyecto final · Upskilling Backend Java

QUÉ DECIR (? = HACER CLIC)

“Empezamos con un endpoint y terminamos con dos microservicios con sus bases de datos, empaquetados, desplegados, observados y documentados, que saben comportarse cuando algo falla.”

“Muchas gracias por acompañarnos, de parte de todo el equipo arka-litos. Con gusto respondemos sus preguntas.”

QUÉ HACER (NO SE DICE)

- Quien orquesta recibe las preguntas y se las pasa a quien presentó esa parte.